



COMPUTER ORGANIZATION & ASSEMBLY LANGUAGE(CS-215T)

Lecture by:
Mr. Shakir Karim
Assistant Professor
DoCS&IT, SSUET

Batch
2022F



Topic

Performance Issues and Improvement Techniques



TRENDS IN TECHNOLOGY

We concluded that

A computer consists of

Gates,

Memory Cells,

Interconnections among these elements

The gates and memory cells are constructed of simple electronic components, such as transistors and capacitors.

The integrated circuit exploits the fact that such components as transistors, resistors, and conductors can be fabricated from a semiconductor such as silicon.

Many transistors can be produced at the same time on a single wafer of silicon



TRENDS IN TECHNOLOGY

Earlier, few gates and memory cells could be reliably manufactured and packed together – SSI

As time went on, it became possible to

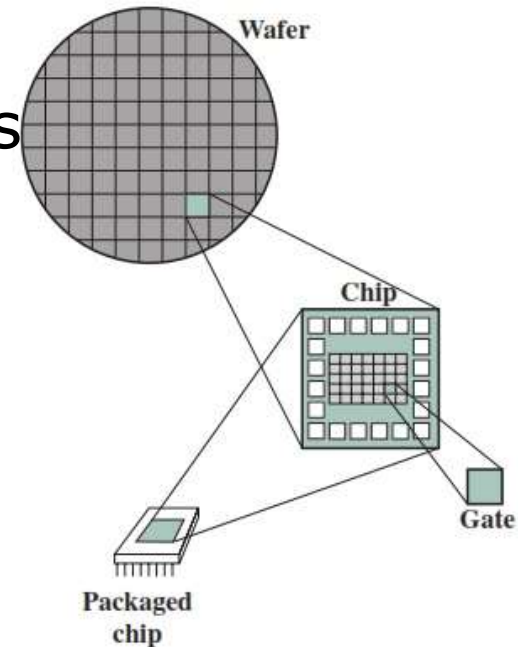
Pack more and more components on the same chip ...

Hence the terms

LSI

VLSI

ULSI





TRENDS IN TECHNOLOGY

Moore observed that the number of transistors that could be put on a single chip was doubling every year the pace continued year after year and decade after decade.

The pace slowed to a doubling every 18 months in the 1970s but has sustained that rate ever since.



The famous Moore's law, propounded by Gordon Moore, cofounder of Intel, in 1965



TRENDS IN TECHNOLOGY

The consequences of Moore's law are profound:

1. The cost of a chip has remained virtually unchanged during this period of rapid growth in density. This means that the cost of computer logic and memory circuitry has fallen at a dramatic rate.

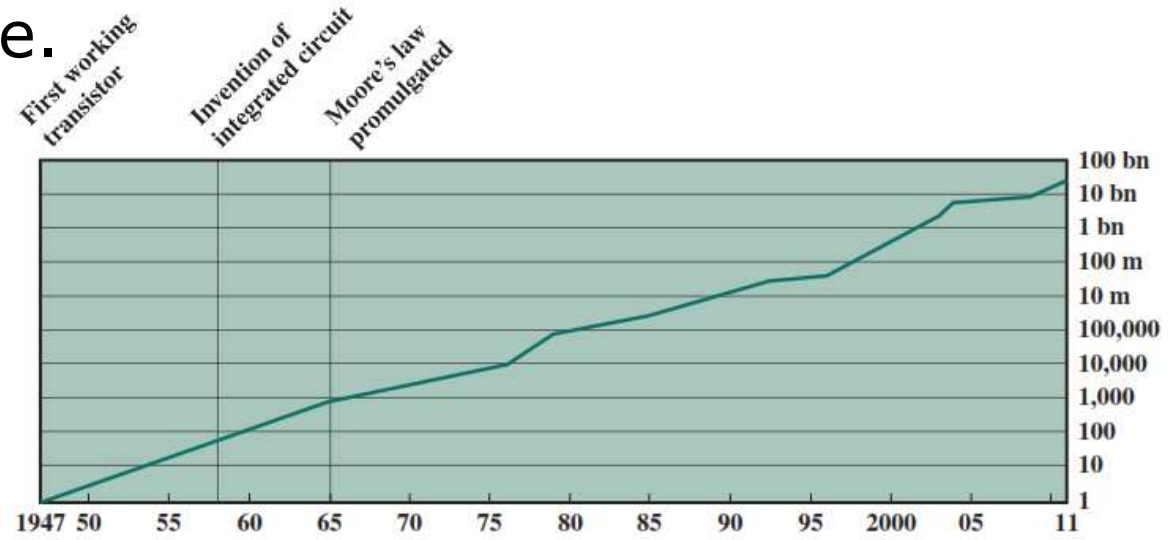


Figure 1.12 Growth in Transistor Count on Integrated Circuits



TRENDS IN TECHNOLOGY

2. Because logic and memory elements are placed closer together on more densely packed chips, the electrical path length is shortened, increasing operating speed.
3. The computer becomes smaller, making it more convenient to place in a variety of environments.
4. There is a reduction in power requirements.
5. The interconnections on the integrated circuit are much more reliable than solder connections. With more circuitry on each chip, there are fewer inter-chip Connections.



TRENDS IN TECHNOLOGY

The concept of family of compatible computers

Characteristics of family includes:

- Similar / Identical Instruction Set
- Similar / Identical Operating Systems
- Increasing Speed
- Increasing Number of IO Ports
- Increasing Memory Size
- Increasing Cost



MICROPROCESSOR—THE SPEED MONSTER

- Remember the ENIAC on a single chip!
- Power of microprocessors seem virtually free
- Providing the power requirements of desktop applications

Videoconferencing

Speech recognition

Image processing

Simulation modeling

3-D rendering

Multimedia authoring

Voice and video annotation of files



ISN'T IT FASCINATING!

Same building blocks as 50 years before

BUT

Sophisticated technology for better performance

Then Why the speed monster?



WHY THE SPEED MONSTER?

Evolution continues at a fast rate

Reducing the distances between circuits

Leading to improved performance 4-to-5 folds every three years

However,

the other side of the coin is different

Seems cooking food on the candle flame!



PERFORMANCE IMBALANCE

Track for McLaren?





TECHNIQUES TO FEED THE MONSTER

- Pipelining
- Branch Prediction
- Superscalar Execution
- Data Flow Analysis
- Speculative Execution

Necessary to execute
many instructions per processor cycle
rather than
to take many cycles per instruction.



BALANCING THE IMBALANCE

Retrieving more bits at one time

- by making DRAMs “wider” rather than “deeper”
- by using wide bus data paths

Making the DRAM interface more efficient

- by including a cache on the DRAM chip

Reduce the frequency of memory access

- by incorporating efficient cache structures between the processor and main memory

Increase the interconnect bandwidth between processors and memory

- by using higher-speed buses and a hierarchy of buses to buffer and structure data flow



IMPROVEMENT IN CHIP ORGANIZATION & ARCHITECTURE

Increase in Hardware speed of processor

Increase the size & speed of caches

Reduce the frequency of memory access

Use Parallelism



OBSTACLES IN IMPROVEMENT

Traditional dominant factors to performance gains are to increase

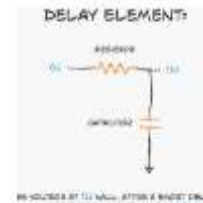
Clock Speed & Logic Density

However, this comes with number of obstacles

Power



RC Delay



Memory Latency & Throughput





TRENDS IN POWER & ENERGY IN ICs

Power is the biggest challenge at present.

Q how should a system architect think about performance, power, and energy?

A three primary concerns

1 - What is the maximum power a processor ever requires?

More power => more current => more voltage drop => device malfunction

Modern processors incorporate voltage indexing and regulate voltage as needed which will decrease performance.



TRENDS IN POWER & ENERGY IN ICs

2- What is the sustained power consumption?

This metric is called Thermal Design Power (TDP) determines the cooling requirement.
manages heat by reducing clock rate

Thereby,

reduces power
and in worse case
activate the thermal overload trip to power down
the chip.



TRENDS IN POWER & ENERGY IN ICs

3- Which metric should be used to compare processors – energy or power?

Energy is better metric

Why?

Since, it is tied to a specific task and the time required to complete that task.

The energy to execute a workload
= $\text{Power}_{\text{Avg}}$ X the execution time for the workload

Energy consumption should be used for comparison instead of the power consumption.



TRENDS IN POWER & ENERGY IN ICs

Example

Processor **A** may have a 20% higher average power consumption than processor **B**, but what happened if **A** executes the task in only 70% of the time needed by **B**?

The energy consumption of **A** will be

$$1.2 \times 0.7 = 0.84$$

which is clearly better.



DEPENDABILITY

Module Reliability:

Mean Time To Failure (MTTF) - reliability measure.

The reciprocal of MTTF is a *rate of failures*, generally reported as failure per billion hours of operation, or FIT (failure in time)

MTTF of 1,000,000 hours equals $10^9/10^6$ or 1000 FIT.



DEPENDABILITY

Service interruption
measured as Mean Time To Repair (MTTR)

$$MTBF = MTTF + MTTR$$

(MTBF = mean time between failures)

What if a collection of modules has exponentially distributed lifetimes (i.e., the age of a module is not important in probability of failure)?

Then, the overall failure rate of the collection is the sum of the failure rates of the modules.



DEPENDABILITY

Example – MTTF & FIT

Answer The sum of the failure rates is

$$\begin{aligned}\text{Failure rate}_{\text{system}} &= 10 \times \frac{1}{1,000,000} + \frac{1}{500,000} + \frac{1}{200,000} + \frac{1}{200,000} + \frac{1}{1,000,000} \\ &= \frac{10 + 2 + 5 + 5 + 1}{1,000,000 \text{ hours}} = \frac{23}{1,000,000} = \frac{23,000}{1,000,000,000 \text{ hours}}\end{aligned}$$

or 23,000 FIT. The MTTF for the system is just the inverse of the failure rate:

$$\text{MTTF}_{\text{system}} = \frac{1}{\text{Failure rate}_{\text{system}}} = \frac{1,000,000,000 \text{ hours}}{23,000} = 43,500 \text{ hours}$$

or just under 5 years.



DEPENDABILITY

Module Availability

measure of the service accomplishment

w.r.t the alternation between the two states of accomplishment and interruption.

For non-redundant systems with repair,

$$\text{module availability} = (\text{MTTF}) / (\text{MTTF} + \text{MTTR})$$



COMPARING ANY TWO SYSTEMS

When a computer is faster than another?

Response time – also referred to as Execution time
-time between start and completion of an event.

Throughput – also referred to as Bandwidth
-the total amount of work done in a given time.

X is faster than **Y** $\Rightarrow$ **X** is **n** times faster than **Y**

$$n = \text{Execution-Time}_Y / \text{Execution-Time}_X$$

$$n = \text{Performance}_X / \text{Performance}_Y$$



COMPARING ANY TWO SYSTEMS

Bandwidth has outpaced latency

A simple rule of thumb:

bandwidth grows by at least the square of the improvement in latency.

Wire delay

scales poorly compared to transistor performance
has become a major design limitation for large ICs
often more critical than transistor switching delay

Scalability – being able to expand memory, the number of processors, and disks



QUANTITATIVE PRINCIPLES OF DESIGN

Parallelism

Parallelism at the system level – Spreading workload of handling requests among the processors and disk.

Parallelism at the data level – spreading of data across many disks for parallel reads and writes

Parallelism at the individual level – parallelism among instructions through pipelining.

Parallelism at digital design level –

carry-lookahead (used by ALU to speed up the process of computing sum)

set-associative caches (multiple banks of memory typically searched in parallel)



QUANTITATIVE PRINCIPLES OF DESIGN

Principal of Locality

Programs tend to reuse data and instructions they have used recently.

Rule of thumb:

a program spends 90% of its execution time in only 10% of the code.

Two types of locality:

Temporal Locality: recently accessed items are likely to be accessed in the near future.

Spatial Locality: items whose addresses are near one another tend to be referenced close together in time.



QUANTITATIVE PRINCIPLES OF DESIGN

Focus on the common cases:

In making a design trade-off,
favor the frequent case over the infrequent case.

This principle applies when determining
how to spend resources,

since the impact of the improvement is higher
if the occurrence is frequent.



LAWS GOVERNING PARALLELISM

In applying the simple principle of common cases, a million dollar question is

what the frequent case is and
how much performance can be improved
by making that case faster?

Two fundamental laws can be helpful

Amdahl's Law
and
Little's Law



AMDAHL'S LAW

proposed by Gene Amdahl in 1967

Statement:

"the performance improvement to be gained from using some faster mode of execution is limited by the fraction of the time the faster mode can be used".

Deals with the potential speedup of a program using multiple processors compared to a single processor



AMDAHL'S LAW

What is speedup?

Speedup is the ratio that tells

how much faster a task will run using the computer with the enhancement as opposed to the original computer

$$\text{Speedup} = \frac{\text{Performance with enhancement}}{\text{Performance without enhancement}}$$



AMDAHL'S LAW

Example:

Consider a program running on a single processor such that a fraction **(1 - f)** of the execution time involves code that is inherently sequential, and a fraction **f** that involves code that is infinitely parallelizable with no scheduling overhead.

Let **T** be the total execution time of the program using a single processor.

Then what will be the speedup using a parallel processor with **N** Processors that fully exploits the parallel portion of the program?



AMDAHL'S LAW

The speedup using a parallel processor with ***N*** Processors that fully exploits the parallel portion of the program will be:

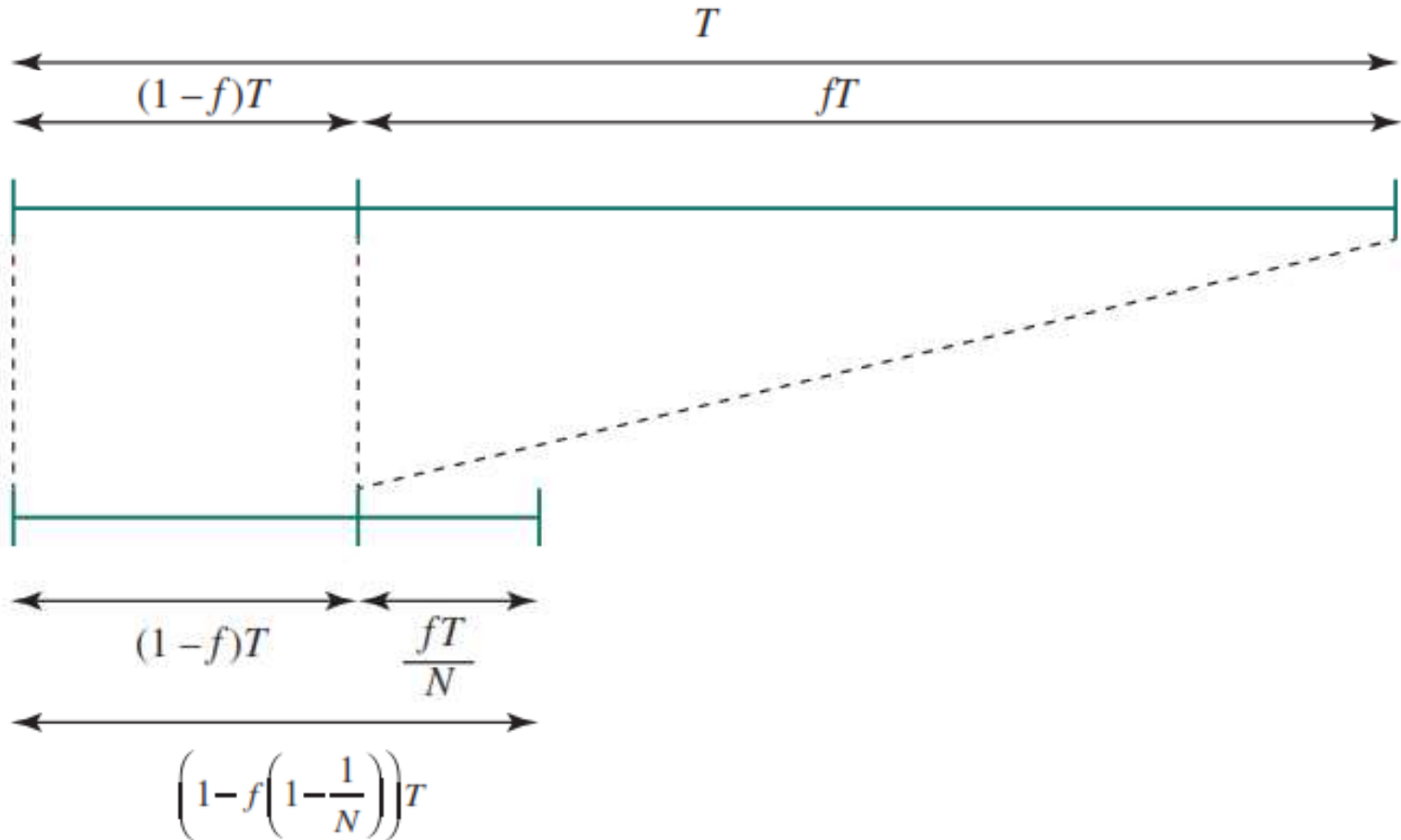
$$\begin{aligned}\text{Speedup} &= \frac{\text{Time to execute program on a single processor}}{\text{Time to execute program on } N \text{ parallel processors}} \\ &= \frac{T(1 - f) + Tf}{T(1 - f) + \frac{Tf}{N}} \\ &= \frac{1}{(1 - f) + \frac{f}{N}}\end{aligned}$$

Two conclusions:

- 1- What if *f* is small ?
- 2- What if $N \rightarrow \infty$?



ILLUSTRATION OF AMDAHL'S LAW





EXAMPLE OF AMDAHL'S LAW

EXAMPLE 2.1 Suppose that a task makes extensive use of floating-point operations, with 40% of the time consumed by floating-point operations. With a new hardware design, the floating-point module is sped up by a factor of K . Then the overall speedup is as follows:

$$\text{Speedup} = \frac{1}{0.6 + \frac{0.4}{K}}$$

Thus, independent of K , the maximum speedup is 1.67.



EXAMPLE OF AMDAHL'S LAW

EXAMPLE

Implementations of floating-point (FP) square root vary significantly in performance. Suppose FP square root (FPSQR) is responsible for 20% of the execution time of a critical benchmark on a machine. One proposal is to add FPSQR hardware that will speed up this operation by a factor of 10. The other alternative is just to try to make all FP instructions run faster; FP instructions are responsible for a total of 50% of the execution time. The design team believes that they can make all FP instructions run two times faster with the same effort as required for the fast square root. Compare these two design alternatives.

Improving the performance of the FP operations overall is better because of the higher frequency. ■



LITTLE'S LAW

A fundamental and simple relation with broad applications is Little's Law

Consider a steady state system to which Items arrive at an average rate of λ items per unit time.

Items stay in the system an average of ***W*** units of time (i.e., average response time).

Finally, there is an average of ***L*** units in the system at any one time.

Little's Law relates these three variables as

$$\mathbf{L} = \lambda \mathbf{W}$$



LITTLE'S LAW

Under steady state conditions,
the average number of items in a queuing system
equals
the average rate at which items arrive
multiplied by
The average time that an item spends in the
system.

This relationship requires very few assumptions.

No need to know:

What the service time distribution is?

What the distribution of arrival times is?

Which order (or priority) items are served in?